

CUU 1.2 - Registrar Pago

Nivel: usuario

Alcance: sistema

Estructura: sin estructurar

Meta: cobrar venta

Actor primario: cajero

Otros actores: cliente

Precondición (de negocio): se ha comprado un artículo

Precondición (de sistema): el cliente posee "venta/s" pendiente/s

Disparador: cajero recibe el pago de un artículo

Flujo de sucesos

- **Camino básico:**
 1. El cliente abona el importe total de la venta al cajero.
 2. El cajero selecciona venta como pagada, el sistema registra el pago.
- **Camino alternativo:** <no analizado>

Fragmento de código

Capa de datos

```
public void add(Sale venta) throws SaleException {
    PreparedStatement stmt = null;
    Statement transaccion = null;

    try {
        transaccion =
            FactoryConnection.getInstance().getConn().createStatement();
        transaccion.execute("start transaction");

        stmt =
            FactoryConnection.getInstance().getConn().prepareStatement(
                "insert into sales "
                + "(emission_date, sale_amount, username, "
                + "cancellation_date, payment_date, withdrawal_date) "
                + "values (?, ?, ?, ?, ?, ?)"
                , PreparedStatement.RETURN_GENERATED_KEYS
            );
        stmt.setTimestamp(1,
            this.toTimestamp(venta.getfEmission()));
        stmt.setDouble(2, venta.getTotal());
        stmt.setString(3, venta.getCliente().getUsername());
        stmt.setTimestamp(4,
            this.toTimestamp(venta.getfCancelacion()));
        stmt.setTimestamp(5,
            this.toTimestamp(venta.getfPago()));
```

```

        stmt.setTimestamp(6,
this.toTimestamp(venta.getfRetiro()));

        stmt.executeUpdate();
        ResultSet primaryKey = stmt.getGeneratedKeys();

        if (primaryKey != null && primaryKey.next()) {
            venta.setNroVenta(primaryKey.getInt(1));
        }

        for (Line linea : venta.getLineas()) {
            lineaVentaData.add(linea, venta.getNroVenta());
        }

        transaccion.execute("commit");
    }
    catch (SQLException doRollback) {
        try {
            transaccion.execute("rollback");
            throw new SaleException("Error when adding new
sale", doRollback, Level.ERROR);
        }
        catch (SQLException e) {
            throw new SaleException("Error when performing
rollback from adding new sale", e, Level.ERROR);
        }
    }
    catch (DBException e) {
        throw new SaleException("Error when establishing
connection to DB, to add new sale", e, Level.ERROR);
    }
    catch (SaleLineException e) {
        throw new SaleException("Error when adding sale line, to
add new sale", e, Level.ERROR);
    }
    finally {
        try {
            if (stmt != null) {
                stmt.close();
            }
            FactoryConnection.getInstancia().releaseConn();
        }
        catch (SQLException e) {
            throw new SaleException("Error when finishing
adding new sale", e, Level.ERROR);
        }
        catch (DBException e) {
            throw new SaleException("Error when closing
connection to DB, after adding new sale", e, Level.ERROR);
        }
    }
}
}

```

Capa de negocio

```
@SuppressWarnings("unchecked")
    public void registerSale(Sale sale) throws SaleException,
    CartLineException {
        ABMCLineaCarrito abmcLineaCarrito = new
        ABMCLineaCarrito(sale.getCliente());
        Cart cart = sale.getCliente().getMiCarrito();

        // Cart to Sale
        sale.setLineas((ArrayList<Line>) (cart.getLineas().clone()));

        // Empty cart on memory
        cart.setLineas(new ArrayList<Line>());

        sale.setfEmission(LocalDateTime.now());
        ventaData.add(sale);

        // Empty cart on DB after sale is added
        abmcLineaCarrito.deleteAllByCart();
    }
```

Capa de presentación

```
protected void doPost(HttpServletRequest request, HttpServletResponse
response) throws ServletException, IOException {
    Customer currentUser = (Customer)
request.getSession().getAttribute("cliente");
    if (currentUser == null || !currentUser.isAdmin()) {
        response.sendRedirect("iniciarSesion.jsp");
        return;
    }

    ABMCVenta abmcVenta = new ABMCVenta();

    String servletPath = request.getServletPath();

    switch(servletPath) {
    case "/RegistrarPagoServlet/IniciarRegistro":
        response.sendError(405, "Method not allowed");
        break;
    case "/RegistrarPagoServlet/Buscar":
        response.sendError(405, "Method not allowed");
        break;
    case "/RegistrarPagoServlet/RegistrarPago":
        String saleNumber = request.getParameter("saleNumber");
        if (saleNumber == null) {
            response.sendError(400, "Parameter 'saleNumber' is
required");
            return;
        }
    }
```

```

    }

    LocalDateTime paymentDate = null;
    String paid = request.getParameter("paid");
    if (paid != null) {
        paymentDate = LocalDateTime.now();
    }

    try {
        Sale venta =
abmcVenta.getOne(Integer.parseInt(saleNumber));
        venta.setfPago(paymentDate);
        venta.setfRetiro(paymentDate);

        abmcVenta.update(venta);

        request.setAttribute("venta", venta);

request.getRequestDispatcher("/WEB-INF/registrarPago.jsp").forward(request,
response);
    }
    catch (NumberFormatException e) {

response.sendRedirect("../errorPage.jsp?mensaje=Numero de venta
incorrecto");
    }
    catch (DoniaMaryException e) {
        response.sendRedirect("../errorPage.jsp?mensaje="
+ e.getMessage());
    }
    catch (Exception e) {
        new DoniaMaryException("Exception caught", e,
Level.ERROR);

response.sendRedirect("../errorPage.jsp?mensaje=Oops ha ocurrido un
error");
    }
    break;
}

```